

BigClaw AI

Design Basis and Description

DBD-AI-001 Rev. 0

Autonomous Investment Research & Portfolio Management System

System Name	BigClaw AI
Version	March 27, 2026
Author	Curtis Biggs
Hostname	BigClaw
Dashboard	https://bigclaw.grandpapa.net
GitHub	https://github.com/cbiggs1-tech/bigclaw-codebase

Document generated March 27, 2026

Based on live system state from Raspberry Pi 'BigClaw' at 192.168.1.171

Table of Contents

1.	System Description	4
2.	Hardware Platform	5
2.1	Hardware Specifications	5
2.2	Development Access	5
3.	Software Architecture	6
3.1	System Diagram	6
3.2	Codebase	6
3.3	Services	9
4.	Database Schema	10
4.1	Tables	10
5.	Portfolio Management Philosophy	11
5.1	Portfolio Definitions	11
5.2	Portfolio Parameters	11
6.	Decision Engine & Scoring System	13
6.1	Signal Dimensions	13
6.2	Style-Specific Signal Weights	13
6.3	Score-to-Label Mapping	14
6.4	Special Rules	14
7.	Autonomous Trading Logic & Execution	15
7.1	Execution Flow	15
7.2	Position Sizing	15
7.3	Safety Rules	15
8.	Risk Management -- Trailing Stops	16
8.1	Trail Percentages by Portfolio	16
8.2	Stop Lifecycle	16
8.3	Intraday Stop Monitor	16
9.	Scheduled Operations & Automation	17
9.1	OpenClaw Cron Jobs (Weekday)	17
9.2	OpenClaw Cron Jobs (Weekly)	17
9.3	OpenClaw Cron Jobs (Daily)	17
9.4	System Crontab Jobs	18
9.5	Disabled Jobs	18
10.	Data Integrity & Anti-Hallucination Pipeline	19
11.	Data Feeds & External APIs	20
11.1	Market Data & Pricing	20
11.2	Options Flow & Institutional Intelligence	20
11.3	Sentiment & Social	20
11.4	News & Research	20

11.5	ARK Invest Tracking	21
11.6	Economic Calendar & Macro	21
11.7	Weather & Environment	21
11.8	Website Frontend	21
11.9	LLM Providers	21
11.10	Communication	22
11.11	Technical Analysis Libraries	22
12.	User Interfaces	23
12.1	Slack -- Primary Interactive Channel	23
12.2	GitHub Pages Dashboard	23
12.3	Dashboard Data Files	23
13.	Logging, Monitoring & Security	25
13.1	Application Logs	25
13.2	Trade Logs	25
13.3	Intraday Stop Check Logs	25
13.4	API Retry	25
13.5	Configuration Files	25
13.6	Security	25
14.	Key Design Principles	26

1. System Description

BigClaw AI is a fully autonomous investment research agent and paper-trading portfolio manager that runs 24/7 on a single Raspberry Pi 4 Model B. Its purpose is simple and unchanging: gather, synthesize, and analyze market data from every available source -- price action, technicals, fundamentals, sentiment, insider activity, and macro signals -- then act on that intelligence to manage seven distinct style-specific paper portfolios.

System Functions:

- Research -- continuously monitors the market without human intervention.
- Manage -- executes simulated trades inside strict, philosophy-driven rules using Alpaca's paper-trading API (each portfolio begins with \$100,000 virtual cash).
- Protect -- enforces position limits, trailing stops, concentration caps, and a hard "no-trade" rule outside the 10:00 AM-4:00 PM ET weekday window.

BigClaw reports every morning (market overview), every afternoon (portfolio performance), and in real time (trade alerts) via Slack and the public dashboard at bigclaw.grandpapa.net. It improves itself through weekly research sessions, style-compliance audits, and decision-engine refinements.

Important reminder for anyone who inherits or touches this system: BigClaw is strictly paper trading. It has never and will never execute real-money trades. Any future attempt to connect it to a live brokerage account must be treated as a complete redesign.

2. Hardware Platform

BigClaw runs on a Raspberry Pi 4 Model B. This choice is deliberate: it is low-power, silent, always-on, and inexpensive to replace. The entire system -- code, database, logs, and GitHub Pages refresh -- lives on this one device. The local Windows copy and GitHub repository exist only as review snapshots; the Pi is the single source of truth.

Development happens primarily through VS Code Remote SSH (key-based ed25519 authentication). The OpenClaw agent runtime and Claude Code CLI are also installed directly on the Pi.

2.1 Hardware Specifications

Component	Specification
Board	Raspberry Pi 4 Model B Rev 1.4
Processor	Broadcom BCM2711, ARM Cortex-A72 (4 cores @ 1.8 GHz)
Architecture	aarch64 (64-bit ARM)
Memory	8 GB LPDDR4
Storage	256 GB USB SSD (228 GB usable, 8% utilized)
Operating System	Debian GNU/Linux 13 (Trixie)
Kernel	Linux 6.12.62+rpt-rpi-v8 SMP PREEMPT
Hostname	BigClaw
Network	Local LAN at 192.168.1.171, SSH alias `bigclaw`
Power	Continuous operation, auto-restart on failure

2.2 Development Access

Method	Detail
VS Code Remote SSH	Primary development interface from Windows desktop
Claude Code CLI	Installed on Pi via OpenClaw (~/.openclaw/)
SSH from Windows	`ssh bigclaw` (key-based auth, ed25519)

3. Software Architecture

The system is deliberately split into two cooperating codebases so that data collection, trading logic, and interactive conversation stay fast and cheap while heavy reasoning is offloaded to cloud LLMs only when needed.

3.1 System Diagram

```
Raspberry Pi 4B -- "BigClaw"
|
|---- systemd: bigclaw.service (always running)
|   |---- bot.py -> Slack agent + Claude conversation loop
|       |---- agent.py      -- Claude tool-use agent
|       |---- tools/ (15 modules) -- market, charts, orders, social, etc.
|       |---- services/     -- price oracle, output guardrail
|       |---- portfolio.py   -- SQLite ORM for all portfolio data
|       |---- llm_router.py  -- cost-optimized model routing
|       |---- memory.py      -- conversation memory
|
|---- systemd: openclaw-gateway.service
|   |---- OpenClaw agent runtime (WhatsApp/Slack gateway + cron engine)
|       |---- cron/jobs.json  -- 14 scheduled jobs
|       |---- workspace/scripts/ -- 30+ Python scripts
|       |---- workspace/skills/ -- 27 modular agent skills
|       |---- workspace/config/ -- portfolio universes, expert overrides
|       |---- workspace/SOUL.md -- agent personality & analytical mandate
|
|---- System crontab (2 jobs)
|   |---- stop_check.py      -- 15-min trailing stop monitor
|   |---- tsla_watchdog.py   -- TSLA dark pool / options monitor
|
|---- docs/ -> GitHub Pages (bigclaw.grandpapa.net)
|   |---- 10 HTML pages
|   |---- data/ -- 11 JSON files + chart data (auto-refreshed)
|
|---- src/portfolios.db -- SQLite database (source of truth)
```

3.2 Codebase

3.2.1 Core Application -- `~/bigclaw-ai/`

The core application is the always-on Slack bot, the portfolio database, and the public dashboard. It runs as `bigclaw.service` under systemd and is the user-facing side of BigClaw.

Slack Bot (`src/bot.py`) -- Built on `slack-bolt` with Socket Mode (no inbound webhooks, no public ports). The bot listens for direct messages and channel mentions, routes each message through the Claude agent with full conversation memory, and returns the response. Image-based responses (charts) are uploaded as Slack files. The bot requires three tokens: `SLACK\_BOT\_TOKEN`, `SLACK\_APP\_TOKEN`, and `SLACK\_SIGNING\_SECRET`.

Claude Agent (`src/agent.py`) -- Implements a tool-use conversation loop against the Anthropic API. When a user sends a message, the agent passes it to Claude along with the full set of available tools. If Claude invokes a tool, the agent executes it locally, feeds the result back, and continues the loop until Claude produces a final text response. Conversation history is maintained per-channel via `memory.py` so context carries across messages.

Interactive Tools (`src/tools/`) -- 36 tools organized across 10 modules that Claude can invoke during conversation:

Module	Tools	What They Do
`market.py`	`get_stock_quote`, `get_stock_details`	Live price, fundamentals,
`charts.py`	`generate_stock_chart`, `compare_stocks_chart`, `generate_macd_chart`, `generate_rs`	Technical analysis chart
`news.py`	`get_yahoo_news`, `search_financial_news`	Per-ticker and broad fina
`social.py`	`get_x_sentiment`, `get_stocktwits_sentiment`, `search_reddit_stocks`, `get_wsb_tre`	Social sentiment from X/T
`predictio`	`search_polymarket`, `get_polymarket_trending`	Prediction market odds
`portfolio`	`create_portfolio`, `delete_portfolio`, `list_portfolios`, `view_portfolio`, `buy_s`	Full portfolio CRUD and t
`orders.py`	`set_stop_loss`, `set_limit_buy`, `set_limit_sell`, `view_pending_orders`, `cancel_`	Pending order management
`strategy_`	`run_analysis_now`	Trigger immediate decisio
`technical`	Additional technical analysis tools	Extended TA capabilities
`base.py`	`set_report_channel`, `set_autonomous_trading`	System configuration

LLM Router (`src/llm\_router.py`) -- A lightweight cost-optimization layer that routes LLM calls to the cheapest model capable of the task. Routine data summarization goes to Gemini Flash Lite via OpenRouter (~\$0.01/call). Analytical work requiring deeper reasoning is routed to Claude Sonnet (~\$0.20/call). Interactive Slack conversations use Claude Sonnet directly via the Anthropic API. The router exposes `call\_openrouter()` for general calls and `summarize\_with\_flash()` as a convenience wrapper for cheap summarization tasks.

Services (`src/services/`):

- Price Oracle (`price\_oracle.py`) -- Single source of truth for all prices system-wide. In-memory cache with 10-minute TTL. Refuses to serve stale data.
- Output Guardrail (`output\_guardrail.py`) -- Scans every outbound message for `\$XXX.XX` price patterns and cross-checks against the oracle. Corrects any deviation > 2%.

Other components:

- `src/portfolio.py` -- SQLite ORM for all portfolio data (holdings, transactions, snapshots, orders, trailing stops)
- `docs/` -- GitHub Pages website with auto-refreshed JSON data files
- `logs/` -- Application logs (rotated at 5 MB, 2 backups)
- `src/portfolios.db` -- SQLite database (source of truth for all portfolio state)

3.2.2 Automation Layer -- `~/openclaw/workspace/`

The automation layer runs as `openclaw-gateway.service` under systemd. It provides the cron engine that drives all scheduled operations, 30+ standalone Python scripts for data gathering and trading, modular agent skills, and the `SOUL.md` personality file.

Scripts (`scripts/`) -- 30+ standalone Python scripts that handle everything the LLM should not be trusted to do on its own: data gathering (`morning\_data\_gather.py`, `afternoon\_data\_gather.py`), price refresh and dashboard updates (`price\_refresh.py`), autonomous trading (`autonomous\_trader.py`), decision engine scoring (`decision\_engine.py`), options intelligence

(`options\_intelligence.py`), trailing stop management (`trailing\_stop\_manager.py`, `stop\_check.py`), TSLA watchdog (`tsla\_watchdog.py`), ARK tracking (`ark\_itk\_tracker.py`), and portfolio reconciliation. All scripts source credentials from `~/env\_secrets` and log via `bigclaw\_logging.py`.

Configuration (`config/`):

- `portfolio\_universes.json` -- Per-portfolio allowed ticker lists (holdings + candidates)
- `expert\_overrides.json` -- Manual conviction score overrides for specific tickers

SOUL.md -- The agent's personality, analytical mandate, and behavioral rules. This file is loaded into every OpenClaw agent session and defines who BigClaw is. Key directives include:

- Prime Directive: Truth and accuracy above all. Protect the user's financial resources. Objectivity over narrative.
- Analytical Mandate: Lead with conviction and directional bias, then explain. Synthesize conflicting signals explicitly. Be bold and interpretive -- state clear leans, scenarios with rationale, and thesis application. Minimize raw data dumps; prioritize interpretation.
- Personality: Sharp, direct, confident when warranted, brutally honest, forward-looking. Humor auto-adjusts by context (TARS-style calibration: low for alerts, higher for casual chat).
- Anti-Hallucination: Never invent prices, percentages, or news. Always call a data tool before responding to any market query. If tools fail, state "data unavailable" -- never guess.
- Autonomy Guardrail: Never autonomously fix code, edit configs, or debug issues unless explicitly asked. Report problems and wait. This is a hard rule that overrides all other instructions.
- Fact Verification: Pull live data via `stock\_quote.py` (primary), `technical\_analysis.py` (technicals), or web search (fallback) before every analytical response. Cross-verify when possible; flag discrepancies > 2%.
- Mandatory Skill Usage: When asked to fact-check, must load and run the `truth-engine` skill (4 independent AI models through a SIFT framework).

Skills -- 30 modular analytical capabilities the agent can load on demand. 27 are installed in `~/openclaw/skills/` (from the ClawHub marketplace) and 3 are custom workspace skills:

Skill	Description
`backtest-expert`	Expert guidance for systematic backtesting -- parameter robustness testing, slippage modeling, b
`finance-news`	Market news briefings with AI summaries across US/Europe/Japan markets
`financial-planning`	Solopreneur business finances -- budgeting, cash flow, P&L, projections
`fundamental-stock-an	Structured scoring playbook -- quality, balance sheet, cash flow, valuation, peer ranking
`institutional-flow-t	Track 13F filings -- hedge fund and institutional ownership changes, smart money flow
`intellectia-stock-fo	AI-powered entry/exit analysis, target price predictions, probability calculations
`log-analyzer`	Parse, search, and analyze application logs; debug from log files; error pattern analysis
`market-environment-a	Comprehensive global market analysis -- risk-on/off assessment, sector analysis, technical indic
`news-summary`	Fetch and summarize news from trusted international RSS feeds (workspace custom)
`onchain`	Crypto portfolio tracking, market data, CEX history, transaction lookups
`openinsider`	SEC Form 4 insider trading data -- director, CEO, and officer buys/sells
`options-strategy-adv	Options pricing (Black-Scholes), Greeks, strategy P&L simulation, earnings plays
`portfolio-manager`	Portfolio analysis via Alpaca -- allocation, risk metrics, diversification, rebalancing

Skill	Description
`python-dataviz`	Professional data visualization with matplotlib, seaborn, plotly -- static and interactive
`realtime-x-sentiment`	Real-time X/Twitter sentiment polling every 2 hours, alerts on shifts > 5%
`sector-analyst`	Sector and industry performance analysis -- market cycle assessment, rotation patterns
`sec-watcher`	Monitor SEC EDGAR filings for 50+ AI/tech companies -- 8-K, 10-K, 10-Q, insider transactions
`self-improve`	Agent self-diagnosis -- fix configs, prompts, response quality, personality tuning
`skill-vetting`	Security and utility assessment before installing third-party ClawHub skills
`stock-evaluator`	Comprehensive stock evaluation -- valuation, fundamentals, technicals, entry prices, conviction
`stock-strategy-backt`	Backtest trading strategies on historical OHLCV data -- win rate, CAGR, drawdown, Sharpe ratio
`time-series-analysis`	Time series feature engineering, model training, and forecasting
`truth-engine`	Multi-model fact-checking -- 4 independent AI models through SIFT framework with consensus scoring
`uptime-kuma`	Interact with Uptime Kuma monitoring server -- check status, add/remove monitors
`us-market-bubble-det`	Quantitative bubble risk scoring via revised Minsky/Kindleberger framework -- VIX, margin debt,
`yahoo-finance`	Stock prices, quotes, fundamentals, earnings, options, dividends, analyst ratings via yfinance
`afrexai-esg-reportin`	ESG scoring on Environmental, Social, and Governance factors aligned with 2026 disclosure standa
`afrexai-portfolio-ri`	Monte Carlo simulations, Value at Risk, stress tests, drawdown, beta, Sharpe ratio
`afrexai-tax-planning`	Tax-loss harvesting suggestions, entity structure optimization, deduction planning

3.3 Services

Two systemd services run permanently:

Service	Type	Purpose
`bigclaw.service`	systemd (always-on)	Slack bot + Claude agent + 36 interactive tools
`openclaw-gateway`	systemd (always-on)	Agent runtime, cron engine, Slack/WhatsApp gateway

A handful of lightweight crontab jobs (`stop\_check.py` every 15 min during market hours and `tsla\_watchdog.py`) run outside the LLM loop for zero-token-cost monitoring.

4. Database Schema

The database (SQLite) is deliberately simple and self-contained. Everything that matters is here; nothing is hidden in external services. All portfolio state lives in `src/portfolios.db`.

4.1 Tables

```
portfolios
  id, name, investment_style, starting_cash, current_cash,
  report_channel, is_active, purchase_status, created_at

holdings
  id, portfolio_id, ticker, shares, avg_cost, rationale,
  first_bought_at, last_bought_at

transactions
  id, portfolio_id, ticker, action (buy/sell), shares, price,
  total_value, rationale, executed_at

daily_snapshots
  id, portfolio_id, snapshot_date, total_value, cash,
  holdings_value, daily_return

pending_orders
  id, portfolio_id, ticker, order_type (stop_loss/limit_buy/limit_sell),
  trigger_price, shares, status, triggered_at

trailing_stops
  id, portfolio_id, ticker, high_water_mark, hwm_date, trail_pct,
  trigger_price, status (active/triggered/stale), created_at,
  updated_at, triggered_at
```

5. Portfolio Management Philosophy

BigClaw manages seven independent paper portfolios. Each is permanently tied to a distinct investment philosophy modeled after a legendary investor or thematic approach. The decision engine applies unique signal-weighting matrices to each style so that a ticker that scores as a "STRONG BUY" for Momentum Growth may legitimately score as a "HOLD" or "SELL" for Value Picks. This style fidelity is non-negotiable and is the single most important mechanism that prevents the system from drifting into generic "just buy what's hot" behavior.

5.1 Portfolio Definitions

#	Portfolio	Investment Style	Modeled After
1	Value Picks	Deep Value Investing	Benjamin Graham
2	Innovation Fund	Disruptive Innovation	Cathie Wood / ARK
3	Growth Value	Growth at Reasonable Price (GARP)	Peter Lynch
4	Income Dividends	Income / Dividend Growth	Dividend Aristocrats
5	Momentum Growth	Momentum / Aggressive Growth	Quantitative Momentum
6	Nuclear Renaissance	Nuclear Energy / Domain Expertise	Thematic Structural
7	AI Defense & Autonomous	AI Defense / Autonomous Systems	Pentagon Spending Theme

5.2 Portfolio Parameters

Parameter	Value
Starting capital per portfolio	\$100,000 virtual cash
Maximum holdings per portfolio	10
Minimum holdings per portfolio	7 (triggers swap/add if breached)
Maximum single position	20% of total portfolio value (holdings + cash)
Rebalance trim target	18% of total portfolio value
Concentration monitoring	Every 2 hours (Slack warning if > 20%)
Auto-rebalance	First trading day of each month
Cash reserve	Minimum 2% of starting capital retained
Maximum single order	\$25,000

Parameter constraints and expert overrides are defined in configuration files so future maintainers can expand or tighten them without touching core code.

Each portfolio operates within a curated universe of allowed tickers stored in `config/portfolio\_universes.json`. The universe is split into two tiers: holdings (tickers the portfolio currently owns) and candidates (pre-approved tickers eligible for swap-in when the decision engine identifies a stronger alternative). Specific tickers change over time as the engine recommends swaps and the market evolves -- the configuration file is the authoritative, living source for current membership.

Value Picks -- The universe centers on blue-chip stalwarts that Benjamin Graham would recognize: mega-cap consumer staples, financials with fortress balance sheets, healthcare giants, and legacy industrials trading below intrinsic value. These are companies with decades of earnings history, wide moats, and the kind of boring predictability that deep-value investors prize. The decision engine heavily weights P/E, debt-to-equity, and insider activity for this portfolio.

Innovation Fund -- This is the widest universe, deliberately so. It spans disruptive technology across AI, quantum computing,

biotech, fintech, cloud infrastructure, and high-growth SaaS -- companies that are redefining industries or creating entirely new ones. Many of these names are volatile and unprofitable by traditional metrics, which is exactly why the engine zeroes out P/E and debt signals for this style and instead emphasizes revenue growth, relative strength, and momentum crossovers.

Growth Value -- A tightly curated universe of large-cap compounders that deliver strong earnings growth at reasonable valuations -- the "best of both worlds" names that Peter Lynch built his career on. These are household-name technology and financial companies that dominate their markets but still have room to grow. The engine balances technical and fundamental signals roughly equally for this style.

Income Dividends -- REITs, utilities, MLPs, consumer staples, and dividend aristocrats -- companies selected primarily for reliable, growing income streams. The universe favors names with long dividend track records, sustainable payout ratios, and defensive characteristics during market downturns. Bond market signals and dividend yield carry double weight in this portfolio's scoring.

Momentum Growth -- The universe is sector-agnostic and changes the most frequently. It tracks names showing the strongest relative strength, price acceleration, and technical breakouts -- wherever they occur. The engine applies its heaviest momentum weighting here (MACD, SMA crossovers, and relative strength all at 2x) and largely ignores fundamentals. This portfolio has the tightest trailing stop (10%) because momentum names reverse fast.

Nuclear Renaissance -- A focused thematic universe covering the nuclear energy supply chain: uranium miners, fuel processors, reactor designers (both conventional and SMR), and nuclear-adjacent power generators. This is a structural thesis -- the multi-decade buildout of nuclear capacity for AI data centers and grid decarbonization -- so the universe is deliberately small and deep rather than broad.

AI Defense & Autonomous -- Defense primes, autonomous systems developers, drone manufacturers, space companies, and cybersecurity firms positioned to capture the Pentagon's accelerating spending on AI-enabled warfare and autonomous platforms. The universe includes both established contractors and emerging pure-plays. Revenue growth carries double weight here, reflecting the sector's rapid budget expansion.

Universe files can be updated without touching core code. The decision engine's `--rescreen` mode evaluates all candidates in each universe and recommends swaps when a candidate outscores the weakest current holding by 3 or more points.

6. Decision Engine & Scoring System

The analytical heart of BigClaw is `decision\_engine.py`. It evaluates every candidate ticker across 14 signal dimensions (technical, fundamental, sentiment, macro, and override factors) and then multiplies those raw signals by a style-specific weight matrix (0 = ignore, 1 = normal, 2 = double emphasis). The result is a composite score that is mapped to clear action labels.

This weighted, multi-factor approach is why BigClaw stays true to each portfolio's thesis instead of chasing the same momentum names across every style.

6.1 Signal Dimensions

#	Category	What It Measures	Score Ran
1	RSI	Relative Strength Index (14-day)	-1 to +1
2	MACD	MACD/signal line crossovers	-0.5 to +1
3	SMA50	Price vs 50-day simple moving average	-0.5 to +1
4	SMA200	Price vs 200-day simple moving average	-0.5 to +1
5	Cross	Golden cross (50 > 200) or death cross	-0.5 to +1
6	RelStrength	1-month return vs sector ETF	-0.5 to +1
7	EarningsGrowt	Quarterly earnings growth rate	-1 to +1
8	RevenueGrowth	Revenue growth rate	-1 to 0
9	PE	Forward P/E vs trailing P/E direction	-1 to 0
10	DebtEquity	Debt-to-equity ratio (flag if > 100)	-1 to 0
11	ShortInterest	Short float percentage (flag if > 10%)	-1 to 0
12	Insider	Net insider buying vs selling (20 transactions)	-1 to +1
13	BondMkt	Bond market signal (fed funds, yield curve)	Variable
14	ValueOverride	Analyst target discount, RSI oversold, P/B < 1, capitulation volume, expert override	0 to +5
15	DividendYield	Dividend yield level	-1 to +1

6.2 Style-Specific Signal Weights

Each portfolio multiplies signal categories differently. Weight 0 = ignore, 1 = normal, 2 = double emphasis.

Signal	Value	Innovation	Growth	Income	Momentum	Nuclear	Defense
RSI	0.5	0.5	0.5	0.5	1.5	1	1
MACD	0	1	0.5	0	2	1	1
SMA50	0	1	0.5	0.5	2	1	1
SMA200	0.5	0.5	1	0.5	1.5	1	1

Signal	Value	Innovation	Growth	Income	Momentum	Nuclear	Defense
Cross	0	1	0.5	0	2	1	1
RelStrength	0	1.5	1	0	2	1.5	1.5
EarningsGrowth	1	0.5	2	1	0.5	1	1
RevenueGrowth	0.5	2	1.5	0.5	0.5	1.5	2
PE	2	0	1.5	1	0	0.5	0.5
DebtEquity	2	0.5	1	1.5	0	1	1
ShortInterest	1	0.5	1	0.5	1	1	1
Insider	2	0.5	1.5	1	0.5	1.5	1.5
BondMkt	1	0.5	0.5	2	0	0.5	0.5
ValueOverride	2	0	1	1	0	1	0.5
DividendYield	1.5	0	0.5	2	0	0	0

6.3 Score-to-Label Mapping

Score	Label	Action
>= 3	STRONG BUY / ADD	Buy candidate
1 to 2	BUY / HOLD	Hold or add on dip
0	HOLD	No action
-1 to -2	WATCH / CAUTION	Monitor closely
<= -3	SELL / TRIM	Trim or exit

6.4 Special Rules

Special rules baked into the engine enforce discipline:

- Bond headwind override: If bond market signal <= -2 and ticker score is marginal (1-2) with weak value override, score is forced to 0 (no buy).
- Value override cap: If raw technical score <= -5 and value override is < 3, the override is prevented from rescuing the ticker above -3 (still flagged as sell/trim).
- Rescreen mode: When run with `--rescreen`, the engine also scores candidate tickers from portfolio universes and generates swap recommendations (candidate must outscore weakest holding by >= 3 points).

7. Autonomous Trading Logic & Execution

Trading is orchestrated once per day at 10:30 AM ET by `autonomous\_trader.py` (and reinforced by the intraday stop-check script). The flow is intentionally sell-first: liquidate weak positions and triggered stops before deploying cash into new opportunities. This prevents over-allocation and ensures cash is always available for the best opportunities.

7.1 Execution Flow

1. Sync -- Reconcile SQLite DB with Alpaca paper account (positions + cash)
2. Analyze -- Run `decision\_engine.py --json --rescreen` to generate signals
3. Monthly rebalance -- On the first trading day of each month, trim any position exceeding 20% of portfolio value back to 18%
4. Sells first -- Free cash before buying:
 - Score ≤ -5 : SELL ALL (100% of position)
 - Score ≤ -3 : TRIM (50% of position)
 - Swap recommendation: SELL ALL (replace with stronger candidate)
5. Trailing stop check -- Execute any triggered trailing stops (between sells and buys)
6. Buys -- Deploy available cash:
 - Score ≥ 3 required to buy (normal mode)
 - Score ≥ 2 if portfolio is $> 60\%$ cash (deployment mode)
 - Score ≥ 1 if part of a swap recommendation
7. Post-trade sync -- Re-compare DB positions against Alpaca after all trades complete. Only mismatches that persist after execution are reported. This eliminates false alerts from pre-trade discrepancies that the trades themselves resolve.
8. Summary -- Post execution report to Slack

7.2 Position Sizing

Score	Allocation	Typical Size
≥ 5	12% of starting capital	~\$12,000
3 to 4	8% of starting capital	~\$8,000
1 to 2 (swap/deploy)	5% of starting capital	~\$5,000

7.3 Safety Rules

These rules must never be relaxed:

- Trading window: Weekdays, 10:00 AM - 4:00 PM ET only. Enforced at cron level, script level, and Alpaca execution level.
- Cash reserve: Minimum 2% of starting capital retained per portfolio
- Max single order: \$25,000
- Round-trip limit: Max 3 round-trips per ticker per week (prevents churning)
- Max holdings: 10 positions per portfolio
- Min holdings: 7 positions per portfolio (triggers swap/add when below)

- Position concentration: No single position may exceed 20% of total portfolio value (holdings + cash)

8. Risk Management -- Trailing Stops

Every new position automatically receives a style-specific trailing stop. The stop only ever moves upward (high-water-mark ratcheting) -- never down. The `stop_check.py` script runs every 15 minutes during market hours with a single yfinance batch call (near-zero cost). Under extreme stress a one-line `tighten_stops(factor)` call can halve all trail percentages instantly. When a position is sold outside the stop system, the stop record is automatically deactivated. This combination of ratcheting logic + frequent lightweight checks is one of the most important capital-preservation mechanisms in the entire system.

8.1 Trail Percentages by Portfolio

Portfolio	Trail %	Rationale
Value Picks	18%	Trust the thesis, wide band
Innovation Fund	18%	Volatile by nature
Income Dividends	15%	Moderate protection
Growth Value	12%	GARP discipline
Nuclear Renaissance	12%	Thematic structural
AI Defense & Autonomous	12%	Thematic structural
Momentum Growth	10%	Cut losers fastest

8.2 Stop Lifecycle

1. Initialize: When a new holding is added, a stop is created with $HWM = \max(\text{current_price}, \text{avg_cost})$
2. Ratchet: Every 2 hours (price refresh) and every 15 minutes (stop check), if $\text{price} > HWM$, the HWM moves up and the trigger price moves up proportionally
3. Trigger: If $\text{price} \leq \text{trigger_price}$, the position is sold at market
4. Tighten: During market stress, the `tighten_stops(factor)` function can cut all trail bands (e.g., $\text{factor}=0.5$ cuts 18% to 9%). This is designed for circuit-breaker scenarios.
5. Stale cleanup: When a position is sold outside the stop system, the stop is automatically deactivated

8.3 Intraday Stop Monitor (`stop_check.py`)

Lightweight Python script that runs every 15 minutes during market hours via system crontab. Zero LLM token cost.

- Fetches prices for all ~44 held tickers (one `yf.download` call, ~3 seconds)
- Runs full stop cycle: cleanup stale stops, initialize new ones, ratchet HWMs, check triggers
- If a stop triggers: executes Alpaca market sell, updates DB, posts to Slack
- Enforces same 10 AM - 4 PM ET trading window as all other components

9. Scheduled Operations & Automation

All recurring work is defined in OpenClaw's ``cron/jobs.json``. Disabled jobs are explicitly listed so no one accidentally re-enables them.

9.1 OpenClaw Cron Jobs (Weekday)

All times Eastern unless noted.

Time	Job	Model	Purpose
8:55 AM	Morning Da	Gemini FI	Runs <code>`morning_data_gather.py`</code> , writes raw data to <code>`/tmp/bigclaw_morning_data.txt`</code>
9:00 AM	Morning Ma	Claude So	Reads morning data file, produces market analysis + portfolio implications, posts to SI
9:00, 11:0	Price Refr	Gemini FI	Runs <code>`price_refresh.py`</code> -- updates portfolio/market/signal prices, ratchets trailing st
9:05, 11:0	Options In	Gemini FI	Runs <code>`options_intelligence.py`</code> -- pulls max pain, IV rank, bullish/bearish premium, sec
10:30 AM	Daily Auto	Gemini FI	Runs <code>`autonomous_trader.py`</code> -- full decision engine + trade execution cycle
4:25 PM	Afternoon	Gemini FI	Runs <code>`afternoon_data_gather.py`</code> , writes raw data to <code>`/tmp/bigclaw_afternoon_data.txt`</code>
4:30 PM	Afternoon	Gemini FI	Reads afternoon data file, produces portfolio performance report, posts to Slack

9.2 OpenClaw Cron Jobs (Weekly)

Schedule	Job	Model	Purpose
Saturday 8:00 AM	ARK ITK Summary	Gemini Flash Lit	Runs <code>`ark_itk_tracker.py`</code> , tracks ARK Invest transactions
Saturday 8:00 AM	Weekly Research Session	Claude Sonnet	Autonomous deep research into market themes and opportuni
Saturday 9:00 AM	Weekly Style Compliance Aud	Gemini Flash Lit	Audits each portfolio for style drift
Sunday 7:00 AM CT	Weekly Network Security Sca	Gemini Flash Lit	Runs security scan of the Pi
Sunday 9:00 AM CT	Weekly OpenClaw Version Che	Gemini Flash Lit	Checks for OpenClaw updates and model version changes

9.3 OpenClaw Cron Jobs (Daily)

Time	Job	Model	Purpose
------	-----	-------	---------

Time	Job	Model	Purpose
7:00 AM CT daily	Good Morning + Security Check	Gemini Flash Lite	System health check, security scan

9.4 System Crontab Jobs

These jobs run via the Pi's system crontab -- pure Python, zero LLM cost.

Schedule	Script	Purpose
7:30 AM CT, weekd	`daily_ex`	Pre-market full export: sector heatmap, calendar, trades (last 2 days), analysis, news, per
9:00, 10:00, 12:0	`refresh_`	Signals + prices + charts refresh: runs decision engine (`export_signals.py`), macro scanne
Every 15 min, 9 A	`stop_che`	Intraday trailing stop monitor (zero LLM cost)
Every 15 min, 2-9	`tsla_wat`	TSLA dark pool / options flow monitor

9.5 Disabled Jobs

Job	Reason
Tuesday Trading Execution	Replaced by Daily Autonomous Trading
Good Morning Curtis	Replaced by Good Morning + Security Check
Email Check: fixit@grandpapa.net	Paused

10. Data Integrity & Anti-Hallucination Pipeline

LLMs are never allowed to fabricate numbers. Every market report follows a rigid two-step process:

Step 1 -- Data Gather (Python, No LLM)

Pure-Python data-gather scripts (``morning_data_gather.py``, ``afternoon_data_gather.py``) pull every price, fundamental, and statistic and write them to flat text files with clear ``=== SECTION ===`` delimiters. Every number, price, and percentage is captured from live APIs before any LLM is involved.

Step 2 -- Analysis (LLM Reads File Only)

The LLM is instructed to read only that file. The prompt enforces critical rules:

- Every number must appear verbatim in the data file
- No web search, no guessing, no fabrication
- If data is missing, state "data unavailable"

Runtime Guardrails

Two runtime guardrails reinforce this:

- Price Oracle (``src/services/price_oracle.py``): Single source of truth for all prices system-wide. In-memory cache with 10-minute TTL. Refuses to serve stale data.
- Output Guardrail (``src/services/output_guardrail.py``): Scans every outbound message for ``$XXX.XX`` patterns and corrects any deviation > 2% against the oracle.

This pipeline is the reason BigClaw's reports remain trustworthy even when LLMs are involved.

11. Data Feeds & External APIs

11.1 Market Data & Pricing

Source	Library/API	Data Provided
Yahoo Financ	`yfinance` 1.1.0	Price history, fundamentals, earnings dates, dividends, options chain, per-ticker new
Finviz	`finvizfinance` 1.3.	Short interest, insider transactions, analyst ratings, price targets
Alpaca	`alpaca-py` 0.43.2	Paper trade execution, account state, position sync, real-time/extended-hours quotes
FRED	CSV API (free)	Michigan Consumer Sentiment, Personal Saving Rate, Treasury yield curve, credit sprea

11.2 Options Flow & Institutional Intelligence

Source	Method	Data Provided
Unusual	REST API (`un	Options flow alerts (unusual volume/premium), dark pool prints, SPY gamma exposure (GEX), marke

Unusual Whales data is pulled in three ways: (1) twice daily in morning and afternoon data gathers for narrative analysis, (2) every 2 hours via `options\_intelligence.py` which collects per-ticker max pain, IV rank, and bullish/bearish premium for all ~41 held tickers plus sector ETF flow and dark pool blocks, and (3) every 15 minutes by the TSLA watchdog for real-time dark pool and options flow monitoring. The options intelligence flat file is automatically appended to the morning and afternoon data gather files so the LLM can reference it in reports.

11.3 Sentiment & Social

Source	Method	Data Provided
X/Twitter	Apify scraper API	Real-time ticker sentiment from financial Twitter
StockTwits	Apify scraper (fallback)	Retail sentiment per ticker
Reddit (WallStreetBets)	Direct API	Trending tickers, retail sentiment
CNN Fear & Greed Index	JSON endpoint (free)	Composite market sentiment score (0-100), used as contrarian indicator

11.4 News & Research

Source	Method	Data Provided
Motley Fool	RSS feed (`feedparser`	Investing and market news via four RSS feeds
Yahoo Finance New	`yfinance`	Per-ticker headlines pulled alongside price data

Source	Method	Data Provided
Brave Search	REST API (API key)	On-demand web search for fact-checking, catalyst verification, and research que
SEC EDGAR	REST API (`edgartools`)	Insider trades (Form 4), annual reports (10-K), regulatory filings
Polymarket	Gamma API (free)	Prediction market odds on economic/political events

11.5 ARK Invest Tracking

Source	Method	Data Provided
CathieArk.com	Web scrape	Daily ARK fund trade notifications -- buys, sells, position changes acr
ARKFunds.io	REST API (free)	Structured ETF trade data from ARK's daily disclosures (primary source)
YouTube / ARK	YouTube Data API + `youtube-trans	Weekly "In the Know" video -- auto-discovered, transcribed, and summari

ARK tracking feeds the Innovation Fund portfolio directly and is summarized weekly via the Saturday ARK ITK cron job.

11.6 Economic Calendar & Macro

Source	Method	Data Provided
Federal Reserve	FOMC calendar scrape	Interest rate decision dates and policy meeting schedule
Bureau of Labor Statistics	Schedule scrape	CPI inflation and Nonfarm Payrolls release dates
Bureau of Economic Analysis	Schedule scrape	GDP release schedule -- quarterly growth data
Yahoo Finance (Earnings)	`yfinance`	Next earnings dates and EPS estimates for portfolio holdings

11.7 Weather & Environment

Source	Method	Data Provided
Open-Meteo	REST API (free, no auth)	Daily weather forecasts for Alvarado, TX -- included in morning briefings

11.8 Website Frontend

Technology	Purpose
Chart.js	Portfolio performance line charts (CDN via jsDelivr)
TradingView	Embedded symbol overview widget for live ticker lookup
Google Fonts (Inter)	Typography across all dashboard pages
GitHub Pages	Static site hosting -- files pushed from Pi via git, zero infrastructure cost

11.9 LLM Providers

Provider	Models Used	Purpose	Cost Model
Anthropic (direct)	Claude Opus 4.6	Interactive development via Claude Code	Pro subscription
Anthropic (direct)	Claude Sonnet	Interactive Slack conversations	Pro subscription
OpenRouter	Claude Sonnet 4.6	Morning analysis, weekly research (high reasoning)	~\$0.20/call
OpenRouter	Gemini 3.1 Flash Lite	Data gather, price refresh, trading (low cost)	~\$0.01/call
OpenRouter	Gemini 2.5/2.0 Flash Lite	Fallback models	~\$0.005/call

Fallback chain: Gemini 3.1 Flash Lite -> Gemini 2.5 Flash Lite -> Gemini 2.0 Flash Lite

11.10 Communication

Service	Purpose
Slack (Bot Token + App Token)	Primary interface -- receives reports, trade notifications, interactive queries
Discord (Webhook)	Error alerts, synchronized reports

11.11 Technical Analysis Libraries

Library	Version	Purpose
`ta`	0.11.0	RSI, MACD, SMA, Bollinger Bands, technical indicators
`pandas-ta`	0.4.71b	Extended technical analysis
`pandas`	3.0.0	Data processing, time series
`numpy`	2.2.6	Numerical computation
`matplotlib`	3.10.8	Chart generation (performance, MACD, RSI, Bollinger, Monte Carlo)
`ffn`	1.1.2	Financial analysis (Sharpe ratio, max drawdown, risk metrics)

The complete, browsable list of all data sources with links and descriptions is maintained on the dashboard's Sources page at bigclaw.grandpapa.net/sources.html.

12. User Interfaces

12.1 Slack -- Primary Interactive Channel

The Slack bot runs as ``bigclaw.service`` and exposes 36 interactive tools that the Claude agent can invoke during conversation. Users interact via direct message or channel mention; the agent maintains per-channel conversation memory. The full tool inventory is documented in Section 3.2.1.

12.2 GitHub Pages Dashboard

Hosted on GitHub Pages at `bigclaw.grandpapa.net` behind Cloudflare Access. Data is refreshed on two schedules:

- 5× daily (9 AM, 10 AM, 12 PM, 2 PM, 4:30 PM CT) via ``refresh_all.sh``: signals, prices, macro, charts
- 1× daily pre-market (7:30 AM CT) via ``daily_export.sh``: sector heatmap, calendar, trades, analysis, news, performance chart

Both scripts alert Slack on failure so stale data is never silent.

Page	File	Purpose
Home	<code>`index.html`</code>	Market overview, portfolio summary cards
Dashboard	<code>`dashboard.`</code>	Today's Planned Actions (sells and strong buys only), portfolio cards, sector rotation heatmap, up
Portfolio	<code>`portfolio.`</code>	Individual portfolio deep dive -- holdings table, metrics, P&L
Signals	<code>`signals.ht`</code>	Single source of truth for all signal categories: action required, watch/caution, opportunities, h
Ticker	<code>`ticker.htm`</code>	Individual ticker detail with candlestick, MACD, RSI, Monte Carlo charts + signal context
Chart Det	<code>`chart-deta`</code>	Portfolio performance comparison chart
Sources	<code>`sources.ht`</code>	Data source documentation, cron schedule reference
Privacy	<code>`privacy.ht`</code>	Privacy policy
Nav	<code>`nav.html`</code>	Shared navigation component
Styles	<code>`styles.htm`</code>	Shared CSS

Design rule: The dashboard shows only actionable items (planned trades). All detailed signal analysis lives on the signals page. This avoids redundancy and ensures the signals page is authoritative.

12.3 Dashboard Data Files

All JSON data in ``docs/data/`` is auto-refreshed and pushed to GitHub Pages:

File	Contents	Refresh
------	----------	---------

File	Contents	Refresh
`signals.json`	Decision engine output: scores, labels, reasons for all held tickers	5×/day
`portfolios.json`	All 7 portfolios with holdings, current prices, returns	5×/day
`market.json`	S&P 500, Dow, NASDAQ, VIX + sector rotation heatmap (11 GICS sectors)	5×/day (indices), 1×/day (s
`macro.json`	Macro indicators, bond yields, sentiment, sector performance, VIX, verdi	5×/day
`charts/*.json`	Per-ticker OHLCV, MACD, RSI, Monte Carlo for ~50 held tickers	5×/day
`trades.json`	Recent trade history (last 2 trading days)	1×/day
`calendar.json`	Upcoming economic events (FOMC, CPI, NFP)	1×/day
`analysis.json`	Macro market scanner report (markdown)	1×/day
`news.json`	Financial news headlines (Motley Fool RSS)	1×/day
`options_flow.json`	Per-ticker max pain, IV rank, bullish/bearish premium; sector ETF flow;	Via options_intelligence.py
`metadata.json`	Last update timestamps, export status	Every refresh
`performance_chart`	Portfolio performance comparison chart	1×/day

Note: `price\_refresh.py` preserves sector data in `market.json` when refreshing index prices -- it merges rather than overwrites.

13. Logging, Monitoring & Security

Everything is logged. ERROR/CRITICAL entries are mirrored to Discord. API calls use automatic retry logic. Secrets live only in `~/env_secrets`` (never in git). SSH is key-only, `fail2ban`` is active, unattended upgrades run, and a weekly security scan is part of the Sunday routine. The GitHub repo is intentionally a read-only review copy -- no secrets ever leave the Pi.

13.1 Application Logs

All scripts log via `bigclaw_logging.py``:

- Log file: `~/bigclaw-ai/logs/bigclaw.log`` (rotated at 5 MB, 2 backups)
- Format: `YYYY-MM-DD HH:MM:SS [bigclaw.module_name] LEVEL: message``
- Discord alerts: ERROR and CRITICAL level messages are auto-posted to Discord webhook

13.2 Trade Logs

- File: `~/openclaw/workspace/logs/trades.log``
- Format: `YYYY-MM-DD HH:MM ET | ACTION | portfolio | ticker | details``
- Actions logged: BUY, SELL, STOP-SELL, REBALANCE-SELL, SWAP, TRIM, SKIP, ERROR

13.3 Intraday Stop Check Logs

- File: `~/bigclaw-ai/logs/stop_check.log``
- Frequency: Every 15 minutes during market hours

13.4 API Retry

All external API calls use `bigclaw_retry.py`` for automatic retries on transient failures (configurable attempts, delay, and label for logging).

13.5 Configuration Files

File	Purpose
<code>~/env_secrets`</code>	All API keys: Alpaca, Anthropic, Slack, Discord, X/Twitter (Apify), O
<code>~/openclaw/openclaw.json`</code>	Agent runtime config: models, channels, gateway settings
<code>~/openclaw/cron/jobs.json`</code>	All 14 scheduled job definitions with schedules, models, prompts
<code>~/openclaw/workspace/config/portfolio_universes`</code>	Per-portfolio allowed ticker lists (holdings + candidates)
<code>~/openclaw/workspace/config/expert_overrides.js`</code>	Manual conviction score overrides for specific tickers
<code>~/openclaw/workspace/SOUL.md`</code>	Agent personality, analytical mandate, behavioral rules
<code>/etc/systemd/system/bigclaw.service`</code>	Systemd service definition for the Slack bot
<code>~/bigclaw-ai/.env`</code>	Application environment variables (Slack tokens, API keys)
<code>~/bigclaw-ai/.gitignore`</code>	Ensures secrets, databases, and caches are never committed

13.6 Security

- All API keys stored in `~/env\_secrets` (not in git, not in any tracked file)
- `.env` and `\*.db` files are in `.gitignore`
- SSH access via ed25519 key authentication only
- Weekly automated network security scan (Sunday 7 AM)
- Daily security check as part of morning routine
- `fail2ban` running for brute-force protection
- Unattended upgrades enabled for security patches
- GitHub codebase repo is a read-only review copy -- no secrets ever pushed

14. Key Design Principles (Guidance for Anyone Who Comes After)

These eight principles are the operating manual. Violate them at the risk of breaking the system's integrity:

1. The Pi is the source of truth. All execution, state, and decisions happen on the Raspberry Pi. Local copies and GitHub are snapshots only.
2. Zero-token where possible. Data gathering, price refreshes, trailing stop checks, and trade execution are pure Python -- no LLM tokens consumed. LLMs are reserved for synthesis, analysis, and interactive conversation.
3. Style fidelity above all. Each portfolio's signal weights are sacred. A stock that's a BUY for Momentum may be a HOLD for Value. Do not apply a generic scoring model across styles.
4. Consistent trading window. No trades before 10:00 AM ET or after 4:00 PM ET, weekdays only. Every component must enforce this -- at cron level, script level, and Alpaca execution level.
5. Anti-hallucination by design. Data files first, LLM second. The 2-step pipeline (Python data gather -> LLM analysis of flat files) prevents fabricated numbers. The Price Oracle and Output Guardrail are non-optional.
6. Sell before buy. The autonomous trader always executes sells first to free cash, then buys. This prevents over-allocation and ensures cash is available for the best opportunities.
7. Stops only move up. Trailing stop high water marks ratchet upward on price increases but never decrease. This is the primary capital-protection mechanism. This locks in gains progressively.
8. Log everything. Every trade, every API call, every decision is logged with timestamps and rationale. Post-mortem analysis depends on it.

This document, the `ARCHITECTURE.md` in the repo, the `SOUL.md`, and the live Pi itself are the complete specification. If you are reading this because you are taking over maintenance, start by running the weekly style-compliance audit and the security scan, then read the logs. The system is designed to be understandable, auditable, and safe -- as long as these principles are respected.